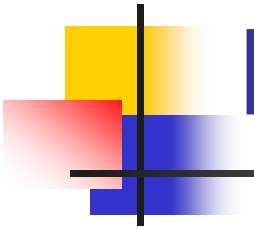
DATA STRUCTURES AND ALGORITHMS

Lecture Notes 4.1

Pradit Pitaksathienkul

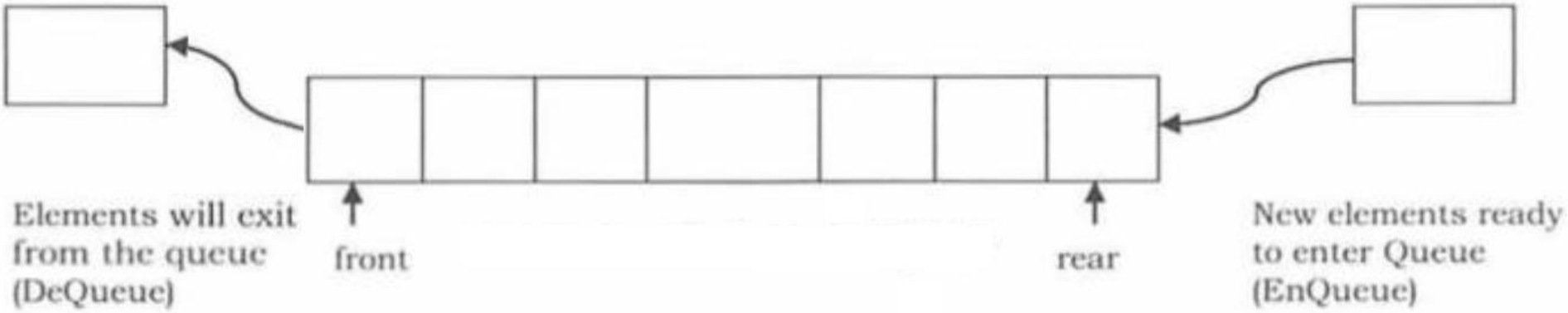


ROAD MAP

- Abstract Data Types (ADT)
  - The List ADT
    - List implementation of lists
    - Linked list implementation of lists
  - The Stack ADT
    - Linked list implementation of stacks
    - List implementation of stacks
  - **The Queue ADT**
    - **List implementation of queues**
    - Linked List implementation of queues

THE QUEUE ADT

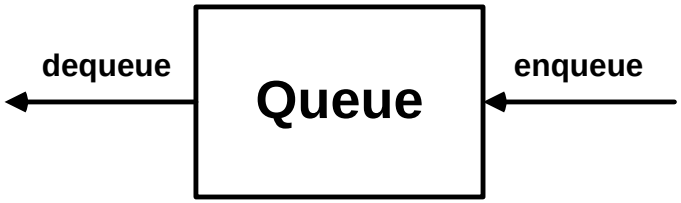
- A queue is a list
  - insertions at one end
  - deletions at other end

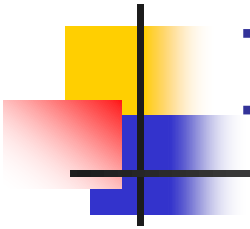


Operations ?

THE QUEUE ADT

- Operations
  - Enqueue
    - insert an element at the end of the list (rear)
  - Dequeue
    - delete the element at the start of the list (front)
  - IsEmpty
    - check whether the queue has an element or not





Implementation of Queue ADT

- Linked list implementation of queues
  - keep two pointers head and tail
- List implementation of queues (Python)
  - keep positions of front and rear
  - keep track of the size of queue
  - data type list in python

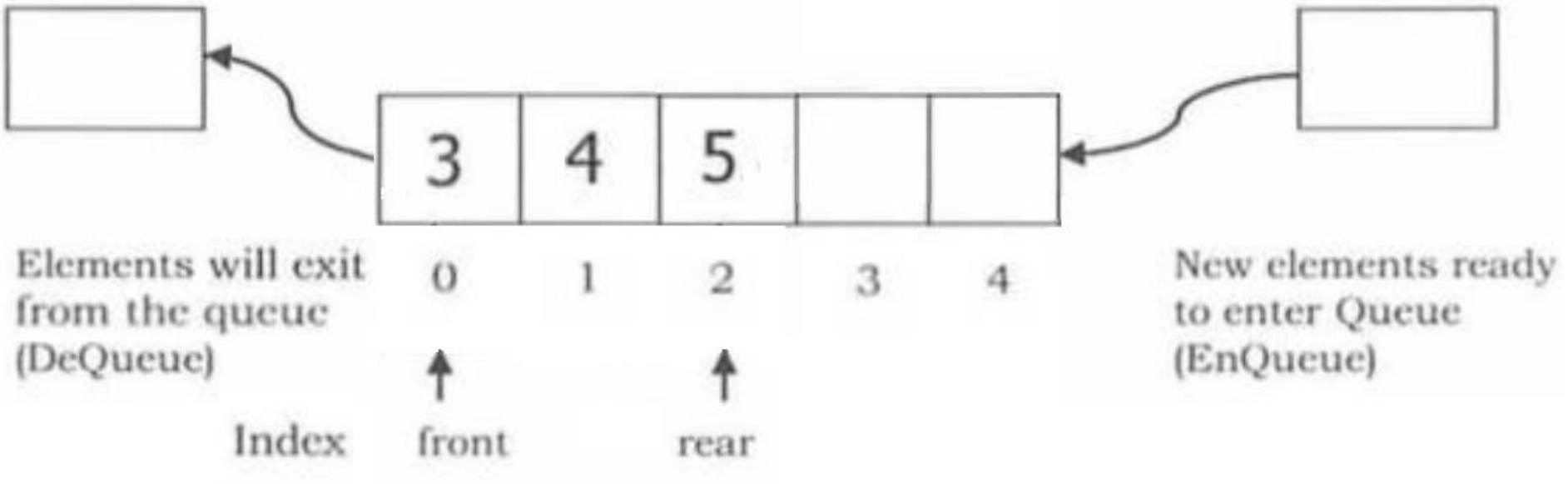
List Implementation of Queues

- Enqueue → Queue [back] = X
  Increment back
  Increment queueSize
- Dequeue → item = Queue[front]
  Decrement queueSize
  Decrement front
  return item

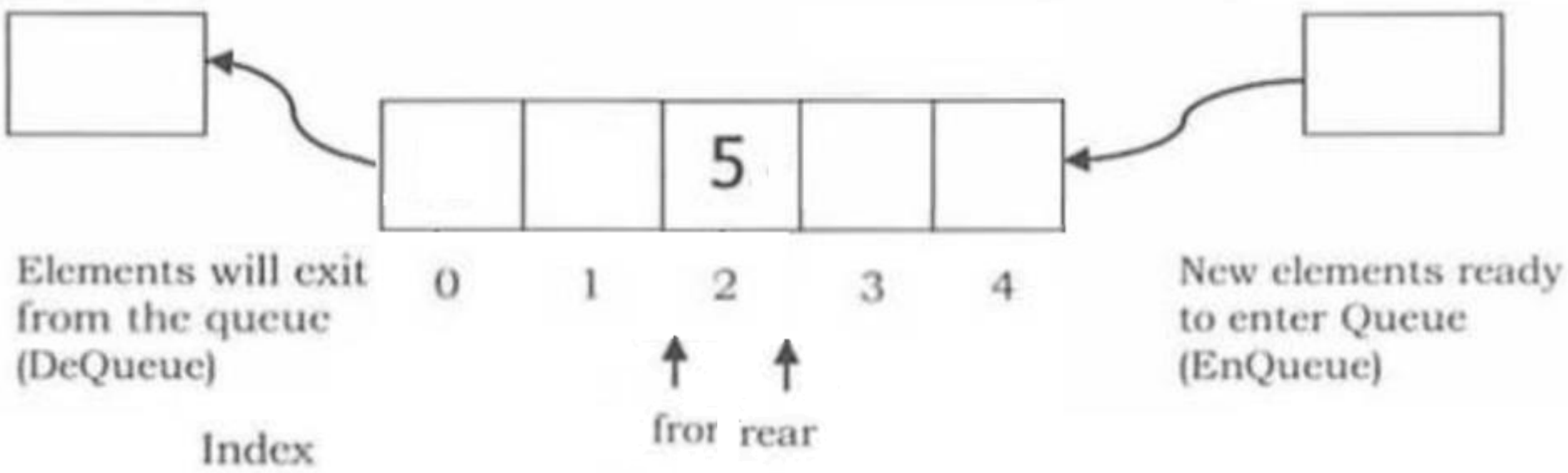
```
class Queue():
    def __init__(self, capacity):
        self.the_array = [None] * capacity
        self.current_size = 0
        self.front = 0
        self.back = 0
    def enQueue(self, item):
       self.the_array[self.back] = item
       self.back = self._increment(self.back)
       self.current_size += 1
    def _increment(self, index):
       index += 1
       if index == len(self.the_array):
          index = 0
       return index
```

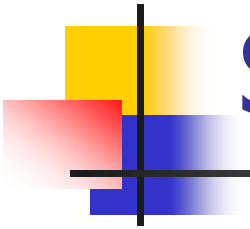
```
def dequeue(self):
    item = self.the_array[self.front]
    self.the_array[self.front] = None
    self.front = self._increment(self.front)
    self.current_size -= 1
    return item

def print_queue(self):
    print("ข้อมูลใน queue: ", end="")
    curr = self.front
    for _ in range(self.current_size):
        print(self.the_array[curr], end=" ")
        curr = self._increment(curr)
    print()
```

```
myqueue = Queue(5)
myqueue.enQueue(3)
myqueue.enQueue(4)
myqueue.enQueue(5)

myqueue.printqueue()
```



```
/* Dequeue - Remove item at the front,

myqueue.deQueue()
myqueue.deQueue()
print("After Dequeue")
```





Special cases of Queues

1. Queue is full, when enqueue
2. Queue is empty, when dequeue

How to solve?